JNTU-GV COLLEGE OF ENGINEERING, VIZIANAGARAM (AUTONOMOUS)
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY GURAJADA, VIZIANAGARAM
DWARAPUDI, VIZIANAGARAM, ANDHRA PRADESH - 535 003.
(A constituent College of JNTUGV & Approved by AICTE, New Delhi)
(Recognised by UGC under section 2(F) & 12(B) of UGC Act 1956)
DEPARTMENT OF INFORMATION TECHNOLOGY

JNTUGV CEV (A)

Dr. Tirimula Rao Benala
Associate Professor
ACM Student Branch Coordinator

Mobile No: 8374033622
Email: btirimula.it@jntugvcev.edu.in

Ref. No.: JNTUGV-CEV/IT/ACM/CIR/2026-27/01

Date: 21-09-2026

CIRCULAR

In collaboration with the ACM JNTU-GV Student Chapter

Sub: Dept. of IT, in collaboration with the ACM JNTU-GV Student Chapter — GenBodha.ai Industry Use-Case Challenge — Award of six months' free platform access to qualifying students — Reg.

All students in B.Tech IV Year (Information Technology), M.Tech (Data Science), and MCA are hereby informed that the Department of Information Technology, in collaboration with the ACM JNTU-GV Student Chapter, is conducting an Industry Use-Case Challenge featuring four real-world problem statements designed by GenBodha.ai. The problem statements are drawn from live industry settings and include detailed hints.

The four use cases are as follows:

S.No	Use Case	What has to be built
1.	**Tier-1 Support Ticket Triage**	Draft replies and route ~1,200 weekly tickets with the agent as human-in-the-loop. The hard part is knowing when to stay quiet.
2.	**Security Questionnaire Autofill**	Answer 200–400 question vendor-security questionnaires from a corpus of past answers, returning the customer's own spreadsheet intact.
3.	**Inbound Lead Qualification**	Score, route and draft first-touch email for ~90 weekly leads within a 30-minute latency budget, with every claim traceable.
4.	**Invoice Exception Handling (3-Way Match)**	Classify ~900 monthly invoice exceptions, propose resolutions and flag duplicate-payment risk with asymmetric error costs.

Award: Every student whose entry, covering any two of the four use cases, is evaluated as successful will receive six months of free access to the GenBodha.ai platform, including its latest LLM models and agents, along with GPU cloud access. If an entry is submitted by a team, each team member will receive access individually.

Entries may be submitted individually or in teams of no more than three students. Each entry must address ANY TWO of the four use cases, chosen by the participant.

What has to be submitted

- A working implementation of both the chosen use cases in a public GitHub repository — either one repository per use case, or a single repository with a clearly separated folder for each — with a README giving the setup steps, how to run the system, and an honest evaluation section.

JNTU-GV COLLEGE OF ENGINEERING, VIZIANAGARAM (AUTONOMOUS)
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY GURAJADA, VIZIANAGARAM
DWARAPUDI, VIZIANAGARAM, ANDHRA PRADESH - 535 003
(A constituent College of JNTUGV & Approved by AICTE, New Delhi)
(Recognised by UGC under section 2(F) & 12(B) of UGC Act 1956)
DEPARTMENT OF INFORMATION TECHNOLOGY

JNTUGV CEV (A)

Dr. Tirimula Rao Benala
Associate Professor
ACM Student Branch Coordinator

Mobile No: 8374033622
Email: btirimula.it@jntugvcev.edu.in

- A presentation of 3 to 5 minutes before the departmental committee, demonstrating both the solutions.
- A short note stating how the system was evaluated, and — equally important — where it abstains or fails.

Basis of evaluation

- Correctness and completeness of the solution against the problem statement.
- Honest evaluation supported by numbers; every use case demands measurement rather than demonstration.
- Handling of failure and abstention — a system that knows when not to answer scores higher than one that always answers.
- Code quality, documentation and reproducibility of the repository.
- Clarity and precision of the presentation.

Schedule

Last date for submission of GitHub repository link & slides	Monday, 28-09-2026
Presentations before the committee	Wednesday, 30-09-2026 at 2.00 PM, Seminar Hall, Dept. of IT

The detailed problem statements, along with the hints for each use case, are attached to this circular and are also available from the Class Representatives. Students are advised to read the hints carefully — in each of these use cases, the credit lies in measured, honest results and in correctly identifying the limits of the system, not in a demonstration that merely appears to work.

Interested students may inform the undersigned, the office bearers of the ACM JNTU-GV Student Chapter, or their respective Class Representatives of their participation. Submissions received after the last date notified above will not be considered.

27/9/2026

Head of the Department
Department of Information Technology

Head of the Department
Department of Information Technology
JNTU-GV College of Engineering
Vizianagaram-535 003
Andhra Pradesh

Copy to:
The Principal, JNTU-GV College of Engineering Vizianagaram (A) — for kind information.
All faculty members, Department of Information Technology.
Class Representatives — B.Tech (IT), M.Tech (Data Science), MCA.
Office Bearers, ACM JNTU-GV Student Chapter.
Notice Board / Department File.

1. **Tier-1 Support Ticket Triage**
   You're contracting for a 400-person B2B SaaS company. Their support team of 9 handles ~1,200 inbound tickets/week through a shared inbox. Median first-response time is 14 hours; the SLA they sell is 4. Roughly 60% of tickets are answered by text that already exists in their 140-article help center — password resets, SSO config, billing-cycle questions, API rate limits. The other 40% genuinely need a human, and a meaningful slice are angry enterprise customers where a wrong automated answer is worse than silence.

   The VP of Support will not accept a bot that replies directly to customers. She will accept a system that drafts a reply and routes the ticket, with the agent as the human in the loop who approves or rewrites. Her success metric is *agent handle time*, not deflection.

   Build it. Show her, with numbers, when your system should stay quiet.

   Hints

- The hard problem is not answering; it's knowing when you can't. Build the abstain path before the answer path.
- Separate "what is this ticket about" from "what is the answer." Different models, different costs, different failure modes.
- Make the answer cite the article ID it came from. If it can't cite, it can't send. This one constraint kills most hallucination.
- Tickets are messy: one email often contains two unrelated problems. Decide early whether you split them or punt.
- Ask what the VP's metric implies. Handle time improves even when the draft is *wrong*, if the wrongness is obvious and the structure is right. That's a real finding — report it.
- Evaluation: label 50 tickets by hand before you write a prompt. You will be tempted to skip this. Don't.

2. Security Questionnaire Autofill

A Series B startup selling to banks and hospitals. Every deal over $50k now arrives with a 200–400 question security questionnaire — SIG Lite, CAIQ, or a bespoke spreadsheet the customer's vendor-risk team invented. Their solutions engineer, one person, spends 6–10 hours per questionnaire. Deals slip by two weeks waiting on him. He has answered most of these questions before, across 60 past questionnaires sitting in a SharePoint folder, plus a policy pack (SOC 2 docs, DPA, architecture overview).

Two constraints from the CISO: (a) an answer that overstates a control is a compliance incident, not a bug; (b) the output has to come back in the customer's original spreadsheet, same rows, same format.

Reduce the 6–10 hours. Be explicit about which questions you refuse to touch.

Hints

- Nearly all your accuracy comes from finding the right prior answer, almost none from writing prose. Spend your time on retrieval.
- Questions that look identical often aren't: "Do you encrypt data at rest?" vs. "Do you support customer-managed encryption keys?" Test your retriever against exactly this trap.
- Yes/No/N.A. columns and the free-text justification are different problems. Don't solve them with one prompt.
- Build an adversary: inject ten questions whose honest answer is "no" into your test set and see whether your system says yes. That number is your headline result.
- Confidence should be a function of retrieval quality, not the model's self-report. Models are bad at knowing they're wrong; retrieval scores are at least measurable.
- Round-tripping the spreadsheet without mangling merged cells and formatting is half the real engineering. Budget for it.

3. Inbound Lead Qualification

A 50-person dev-tools company. Their "Contact Sales" form gets ~90 submissions/week. Two SDRs work them. They currently go oldest-first, which means a Fortune 500 platform team waits behind six students and a competitor doing recon. Analysis of last year's closed-won shows the ideal customer: 200+ engineers, already running Kubernetes, US or EU, not an agency. The company maintains a written ICP rubric nobody consistently applies.

Leads go stale fast — internal data says response within 30 minutes triples conversion versus 4 hours.

Score every inbound lead, route it, and draft the first-touch email. The VP of Sales's real fear is a personalized email containing a fact about the prospect that is not true.

Handle that fear explicitly.

Hints

- A form gives you an email domain and a job title. Everything else is something you went and found — and everything you found can be wrong. Keep the two kinds of fact visibly separate in your data model.
- Scoring against a written rubric is largely deterministic. Ask yourself honestly which parts need a model at all; the answer is smaller than you think, and saying so is a good result.
- Every specific claim in the outbound email should be traceable to a retrieved snippet. Build that trace and then *show* it — it's the demo that wins the room.
- Disqualification is as valuable as qualification, and cheaper. Do it first: competitor domains, free-mail addresses, unsupported geographies.
- 30 minutes is a latency budget. Measure end-to-end wall clock, not just accuracy.
- What happens when research finds nothing? A generic email is an acceptable answer. A fabricated one is not.

4. Invoice Exception Handling (3-Way Match)

A 900-person hardware company. AP processes ~3,000 supplier invoices monthly. The ERP auto-matches invoice → purchase order → goods receipt and clears about 70%. The other 900 land in an exceptions queue worked by four people. Typical causes: quantity received differs from quantity billed, unit price drifted from the PO, the goods receipt hasn't been posted yet, tax computed differently, or the supplier sent the same invoice twice.

Most exceptions are boring and resolvable from the data. A few are duplicate-payment risk — the company paid the same invoice twice last year, for $94k, and the CFO has not forgotten.

Classify each exception, propose the resolution, and draft the supplier's email where one is needed. Your error costs are not symmetric — design for that and say how.

Hints

- The model should never do arithmetic. Compute the match in code, hand the result to the model, let it classify and explain. Test this claim by trying it the other way once — the failure is instructive.
- PDF extraction is the noisiest stage and every downstream error traces back to it. Measure extraction accuracy separately before you measure anything else.
- Duplicate detection is a fuzzy-matching problem, not a reasoning problem. Invoice number, supplier, amount, date window. Use the model only for the ambiguous residue.
- Build your cost matrix before you tune anything: what does a missed duplicate cost versus an unnecessary escalation? Tune the thresholds to that matrix, not to F1.
- Seed your synthetic invoices with known defects so ground truth is free. Include a few that look like duplicates but aren't — same supplier, same amount, different month (subscriptions do this).
- "I'm not sure, here's why, here's what I'd check" is a valid system output and should be a first-class class in your taxonomy, not a fallback.